ChatGPT

集创赛 Robei 赛道小车项目深度分析报告

执行摘要

这份报告基于两类证据完成。第一类是你提供的工程快照 `little_car.zip`，其中包含 `model/`、`build/`、`hdmi_rgb_input/`、`source_review/`、`backup/`、`test/` 等目录，可直接用于代码级分析；第二类是公开可检索到的集创赛与 Robei 相关资料，包括官网通知、分赛区/总决赛答辩要求、高校赛事通知与高校获奖报道，用于还原答辩规则、评分关注点与往届赛道特征。需要特别说明的是：你给出的 GitHub 仓库链接在本次联网核验时返回 404，未能直接抓取仓库页面、README、issues 或 commit history；同时上传的工程快照中也未包含 `.git` 元数据。因此，**本报告对代码本体的分析是强证据，对 GitHub issues/commit 历史的分析只能依据工程内备份包、文件时间戳与模块演化进行“重建式推断”**，并已在文中明确标注。 [1]

从代码本体看，这不是一个“自动路径规划 + PID 闭环 + SLAM/视觉”的复杂智能车，而是一个**基于 FPGA 的多外设并行控制小车系统**。它的核心思路是：用按键或蓝牙编辑一条离散路线队列；用 `route_contol_core` 管理菜单/编辑/运行/障碍停止状态；用 `c5_route_controller` 将“下一条离散命令”变成四轮有符号速度；再由 `c5_motor_frame_driver` 生成 C5 小车的 ASCII UART 控制帧；超声波与 RPLidar C1 提供避障停车信号；`bt_route_uart` 负责手机控制、参数调整、状态回传；`gui_route_renderer + hdmi_top` 则把路线、位置、参数和连接状态实时显示到 HDMI 屏幕上。也就是说，这个项目的真正亮点不在“高阶算法堆叠”，而在**Robei/Verilog 下对多串口、多时序模块、图形界面、路线状态机和传感器避障的系统集成能力**。这恰好非常适合答辩，因为它的工程完整度高、可演示性强、每个模块职责明确。

从算法层面看，仓库内**没有发现 PID、卡尔曼滤波、图像处理链路或机器学习模型**。现有“控制器”本质上是**时间驱动的开环离散控制器**：前进用 `step_length -> step_to_ms()` 映射成动作时长，转弯用固定 `turn_ms`，并通过四轮默认速度参数做静态校准。所谓“传感器融合”也不是概率融合，而是**RPLidar 的扇区最小值 + 连续三圈中值滤波 + 长方形车体安全包络判断**，再与超声波停车信号做逻辑或。这一点在答辩中一定要说清：**项目的目标不是展示最先进智能算法，而是展示基于 FPGA 的可靠控制闭环、实时外设并发与工程可落地性**。如果评委问“为什么不用 PID/SLAM”，最好的回答不是编造，而是坦率说明本项目赛题导向与系统约束决定了采用更稳定、可调试、可在 FPGA 上快速完成验证的方案。

从工程风险和答辩风险看，当前代码快照里至少有四个你必须提前准备说明的点。第一，`route_contol_core` 实际内部数组是 16 项，但某些边界判断仍允许到 32，存在**容量判断与真实存储边界不一致**的问题；第二，`gui_route_renderer` 只展开 8 个命令槽做显示，而路线核心维护 16 个槽，存在**“能执行但未完全显示”**的界面截断风险；第三，路线长度统计按“内部命令数 × 步长”计算，因此**转弯命令也会被计入长度**，这更像“控制步数指标”而不是严格几何里程；第四，障碍停车后会清空活动路线，因此当前系统更像**安全急停 + 重新规划**，而不是“暂停后无缝续跑”。这些点都不是灾难，相反，只要你准备好解释，它们会变成你“理解代码细节很深”的加分项。

就答辩准备而言，公开资料显示，集创赛分赛区/总决赛通常高度重视**技术文档、答辩 PPT、现场演讲、作品演示和评委提问**；一些校内选拔通知给出的典型答辩节奏是 **8 分钟以内汇报 + 5 分钟以内提问** 或 **10 分钟内、其中约 6 分钟展示 + 4 分钟问答**。官方通知还明确指出：技术文档和答辩 PPT 是重要评分依据，内容可包括设计思路、实现逻辑、重要代码、仿真截图等。也就是说，你的答辩策略应该是：**先把系统闭环讲清，再把几个“工程性亮点”讲透，最后主动承认边界与未实现项**。这样比泛泛地说“我们做了智能小车”更有说服力。 [2]

研究依据与未指定项

证据来源

本报告对代码与架构的判断，主要依据你提供的工程快照中的以下路径：

证据类别	主要路径	作用
Robei 原始工程	`model/*.model`、`little_car.proj`、`const/top.const`	判断工程源文件、图形化工程结构、约束来源
当前集成导出	`build/*.v`、`build/top.xdc`、`build/top.bit`	判断当前真正参与综合/实现的 Verilog 与比特流产物
中间阶段导出	`hdmi_rgb_input/*.v`	判断 4 月与 7 月之间的演进版本
阅读副本	`source_review/*.md`、`source_review/verilog/*.v`、`module_manifest.csv`	辅助理解模块职责、规模与依赖
测试与实验	`test/*.test`、`tb/*.v`、`test/README_experiments.md`	判断验证覆盖面与实验关注点
里程碑备份	`backup/*.zip`、`backup/416_hdmi/*`	重建开发时间线与阶段性难点
公开赛事资料	集创赛官网、Robei 官网、高校通知/报道	还原答辩规则、赛道风格与评委关注点

当前无法直接核验的内容

用户给出的 GitHub 仓库链接在本次联网访问时返回 404，公开搜索也未能稳定命中对应仓库页面。因此，下列内容**未能直接从 GitHub 页面核验**：仓库 README 原文、issues 列表、commit message、PR 讨论、发布页。对此，本报告采取的是“保守策略”：凡是直接依赖 GitHub 页面才能成立的结论，一律不强行下断言；凡是能从工程快照本体验证的部分，则按代码快照分析。 [3]

需要明确写给评委的假设

下表建议你在答辩材料的“说明/限制”页直接写出来。这样一旦评委追问，你反而会显得很严谨。

项目	结论	备注
GitHub README	未指定	仓库页面未公开可读；本报告改以 `source_review/README.md` 作为工程说明补充
GitHub issues	未指定	无法直接查看
GitHub commit history	未指定	无法直接查看；本文时间线基于内置备份和文件时间戳重建
Robei 版本	未指定	公开下载页当前可见 Robei 3.5.1，但工程实际使用版本未在仓库中明示 [4]

项目	结论	备注
比特流实现工具	部分未指定	工程中明确存在 Robei 与 Vivado 2025.2 的 RTL 检查脚本；`top.json/top.fasm/top.frame/top.bit` 产物形态提示可能还用到开放实现流，但仓库未附构建脚本，故只能谨慎推断
赛题年份	存在资料冲突	公开资料中 Robei 赛题既出现“可重构机器人设计”，也出现“基于 Robei EDA 工具的 IP 设计”，说明不同年份或不同活动存在题面迭代，应以你报名当届官方发布为准 [5]

与赛事答辩直接相关的公开信息

Robei 官网与高校通知显示，Robei/集创赛长期强调 Robei EDA 的图形化数字设计体验，核心范式是模块、端口、连线的可视化搭建；Robei 官网还宣称其可减少 30%–60% 代码书写量。与此同时，集创赛官方通知明确要求分赛区/总决赛提交技术文档、答辩 PPT 和作品海报，其中**技术文档与答辩 PPT 是重要评分依据**；总决赛成绩通常由**演讲汇报、作品展示、问答**等环节综合确定。 [6]

项目整体架构与模块划分

一句话总览

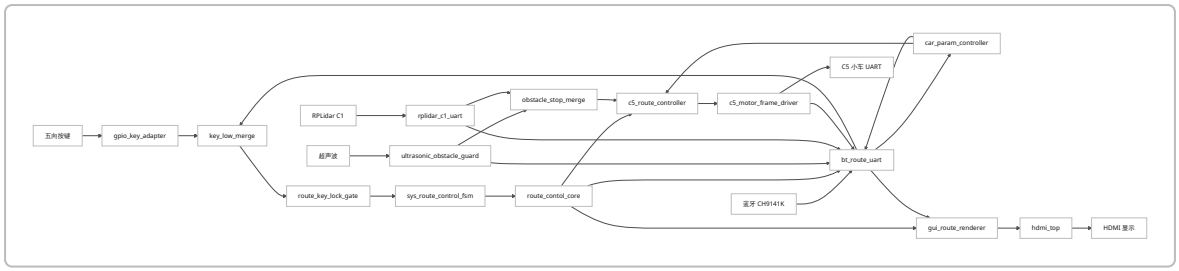
这个项目的真实主线可以概括为：

“输入层统一命令 → 路线状态机决定当前行为 → 运动控制器生成四轮速度 → C5 驱动下发电机帧 → 传感器在旁路给出停车约束 → 蓝牙与 HDMI 做可视化和闭环反馈。”

项目是**模块化、硬件并行、状态机式**的，不是“主程序 + 函数调用”的软件式结构；仓库里也**没有 class 类结构**，主要组织单元是 **Verilog module / function / task / always block**。

顶层数据流与控制流

下面这张图是根据当前主执行链重画的简化版结构图，核心模块全部来自当前快照中的 `build/` 顶层链路。



从 `build/sys_route_gui_fsm_top.v` 可以看到，顶层把 25 个 I/O 端口接给约 22 个实例模块，真正构成主闭环的模块是：`power_on_reset`、`sys_route_control_fsm`、`gui_route_renderer`、`hdmi_top`、`ultrasonic_obstacle_guard`、`gpio_key_adapter`、`buzzer_*`、`bt_route_uart`、`key_low_merge`、`c5_route_controller`、`c5_motor_frame_driver`、`car_param_controller`、`route_key_lock_gate`、`rplidar_c1_uart` 和 `obstacle_stop_merge`。相比之下，`motor_encoder_interface`、`motor_route_controller`、`motor_tb6612_driver` 属于旧链路，当前顶层已经不再使用。

目录与模块角色

结合 `little_car.proj`、`build/` 文件、`source_review/module_manifest.csv` 与当前顶层连接关系，可以把仓库分成如下几层：

层次	关键文件	作用	是否当前主链
工程源	`model/*.model`、`little_car.proj`、`const/top.const`	Robei 图形工程源与约束	是
当前导出	`build/*.v`	最接近当前综合实现的 Verilog	是
中间版本	`hdmi_rgb_input/*.v`	较早导出版本，保留历史演化痕迹	部分
阅读增强副本	`source_review/verilog/*.v`、`*.md`	注释化、可通读版本	辅助
验证	`test/*.test`、`tb/*.v`	Robei 仿真/命令行 testbench	是
历史备份	`backup/*.zip`	里程碑打包	是，但仅作历史依据

如果你在答辩时被问“源文件在哪里”，最稳妥的回答是：

> “Robei 的**真实源头**在 `model/` 和 `little_car.proj`；`build/` 是与当前硬件集成对应的导出 Verilog；`source_review/` 是为了阅读与审核生成的增强副本，不是编译真源。”

关键模块与规模对比

`source_review/module_manifest.csv` 提供了模块体量统计。按代码行数看，本项目最重的模块并不是电机控制，而是 **GUI 与协议**：

模块	代码行数	说明
`gui_route_renderer`	1031	逐像素绘制、文字点阵、路线重建，组合逻辑最大
`bt_route_uart`	977	手机协议、命令译码、回复/状态调度、手动看门狗
`rplidar_c1_uart`	551	雷达收发、节点解析、扇区统计、中值滤波、避障判定
`c5_motor_frame_driver`	397	C5 控制帧生成、UART 发送、停车序列、刷新调度
`route_contol_core`	237	菜单/编辑/运行/停止状态机与路线内存
`c5_route_controller`	235	自动/手动运动控制与四轮速度映射

这张表对答辩非常重要。它说明项目难点其实集中在三件事：**可视化状态表达、串口协议组织、传感器实时解析**，而不是单纯“让车动起来”。

依赖与运行环境

从工程快照可直接看出以下环境信息：

项目	证据	结论
芯片/板卡目标	`build/top.xdc`、`verify_sources.tcl`	AMD/Xilinx Artix-7 `xc7a50tfgg484-1`
时钟	顶层 `clk` + `clk_wiz_0.v`	外部 50 MHz，内部生成 50 MHz 像素时钟与 250 MHz HDMI 串化时钟
开发方式	`little_car.proj`、`model/*.model`	Robei 图形化建模 + Verilog 混合
语法/RTL检查	`source_review/verify_sources.bat`	本地调用 Vivado 2025.2 执行 `synth_design -rtl`
仿真入口	`test/*.test`、`tb/*.v`	Robei 仿真与独立 testbench 并存
实现产物	`build/top.json/top.fasm/top.frame/top.bit`	已完成布局布线并生成 bitstream；实现具体脚本未附，工具链细节未完全指定

这里一个非常值得答辩主动说出的点是：**本项目既保留了 Robei 工程源，又保留了 Verilog 导出与 bitstream 产物，说明团队不是只会“拖模块”，而是能把图形化建模、手写 RTL、仿真验证和硬件下载连起来。** 这会让评委对你的工程能力印象明显更好。

硬件接口与传感器/执行器清单

由 `build/top.xdc` 和顶层端口可整理出实际接口表：

外设	顶层端口	协议/时序	处理模块	说明
五向按键	`key_pin1..5`	低有效输入，带上拉	`gpio_key_adapter` + `key_pulse_filter`	用于菜单/编辑/运行
超声波	`ultrasonic_echo/trig`	10us 触发，60ms 周期测距	`ultrasonic_obstacle_guard`	前向最近障碍检测
蓝牙透明串口	`bt_uart_rx/tx` + `bt_at`	115200 8N1	`bt_route_uart`	手机命令与状态回传；`bt_at` 固定拉高
C5 小车总线	`c5_uart_tx`	115200 8N1	`c5_motor_frame_driver`	向四轮控制器发 ASCII 帧
RPLidar C1	`lidar_uart_rx/tx`	460800 8N1	`rplidar_c1_uart`	扫描启停 + 节点解析
HDMI	`hdmi_clk/d0/d1/d2` 差分对	TMDS	`hdmi_top`	1024×600 实时状态显示
蜂鸣器	`buzzer_out`	GPIO 方波输出	`buzzer_ctrl`	上电/加路线/运行/急停提示

这里还隐藏着两个容易被问到的细节。第一，当前设计里 `c5_uart_rx` 端口虽然保留在顶层，但**没有任何下游解析模块使用它**，说明系统当前是**单向发送式控制**，没有读取 C5 返回状态。第二，蓝牙模块的 `AT` 脚被固定为高电平，意味着工程把 CH9141K 当作**透明串口通道**使用，而不是运行时切 AT 命令模式。

核心算法与实现细节

路线编辑与执行

项目里最容易被误解的一点是“路径规划”。严格来说，这个仓库**没有自动路径规划算法**。它实现的是一种**人工编辑的离散命令队列**。

在 `build/route_contol_core.v` 中，路线命令被编码为 2 bit：

```
localparam [1:0] ROUTE_STOP   = 2'b00;
localparam [1:0] ROUTE_RIGHT  = 2'b01;
localparam [1:0] ROUTE_LEFT   = 2'b10;
localparam [1:0] ROUTE_FORWARD = 2'b11;
```

`route_info[63:0]` 只是把这些命令按 2 bit 一个槽位压平到 64 位总线上，而真正的活动内存是 `route_cmd_mem[0:15]`。这意味着**当前执行核心真实可存的内部命令是 16 条**，不是 32 条。编辑规则也很明确：

- 按“上”只写入一个 `FORWARD`
- 按“左”会写入 `LEFT + FORWARD`
- 按“右”会写入 `RIGHT + FORWARD`
- 按“下”会撤销最近一次“用户动作”，如果最近一步是转向动作，就一次删掉两个内部命令

这一点非常适合答辩中用一句话讲清楚：

> “我们的路线不是按笛卡尔坐标点列保存，而是按‘离散动作序列’保存；转弯在内部会展开为‘转向 + 前进一步’，这样执行控制和路径显示都更简单。”

对应伪代码可以这样说：

```
if state == EDIT:
    if UP:
        push(FORWARD)
    if LEFT:
        push(LEFT)
        push(FORWARD)
    if RIGHT:
        push(RIGHT)
        push(FORWARD)
    if DOWN:
        undo_last_user_action()
```

进入运行态后，`c5_route_controller` 每完成一条动作，就向路线核心送出 `run_step_pulse`，路线核心随后：

1. 把队首命令记入 `old_route_cmd_mem`
2. 若是转向命令，更新 `now_dir`
3. 若是前进命令，更新 `now_x/now_y`

4. 把整个命令队列左移一格
5. `active_route_steps - 1`

伪代码如下：

```
if state == RUN and run_step_pulse and queue_not_empty:
    executed = queue.pop_front()
    history.push(executed)

    if executed == LEFT:
        now_dir = now_dir - 1
    elif executed == RIGHT:
        now_dir = now_dir + 1
    elif executed == FORWARD:
        move(now_x, now_y, now_dir)
```

这里最值得答辩强调的两个“边界问题”

第一，`route_contol_core.v` 中真实数组长度是 16，但 `key_up_pulse` 分支仍用 `active_route_steps < 32 && step_count < 32` 做判断；而左右转分支则改成了 `< 15` 与 `< 16`。这说明**代码曾经历从 32 槽旧实现到 16 槽新实现的重构，但边界条件没有完全统一**。如果评委懂代码，很可能会问到这个点。你的回答应当是：

> “是的，我们在重构时把执行核心改成了 16 项命令存储，但有个别边界判断还保留了旧上限。当前快照里这属于已识别的技术债，答辩后继续修正的优先级很高。”

第二，`total_route_length / complete_route_length / remaining_route_length` 都是**内部命令数 × 步长**。因为转向命令本身也会算入“步数”，它更适合作为**控制进度指标**，不应被解释为严格物理里程。这个问题非常容易被评委敏锐地抓到。最稳妥的表述是：

> “当前的长度条是控制意义上的‘动作量化指标’，不是严格几何路径长度。若后续需要更准确的里程表达，我们会把前进命令和转向命令分开计量。”

运动控制器不是 PID，而是开环定时控制

`build/c5_route_controller.v` 是真正决定车轮速度的核心，但它不是 PID。它做的是三件事：

1. 在自动模式下，根据队首命令决定“前进/左转/右转/停止”
2. 在手动模式下，根据蓝牙 `manual_cmd` 直接覆盖自动路线
3. 根据总速度与四轮默认速度，生成四个轮子的有符号速度

最关键的校准公式是：

```
value = common_speed + wheel_default - CAL_BASE_SPEED;
```

也就是：

```
adjusted_speed = clamp(common_speed + wheel_default - 700, 100, 900)
```

这个式子很重要，因为它体现了项目的**机械一致性补偿思路**：把 700 当作标称平衡点，每个轮子保存一个默认速度修正量，然后和共用速度叠加，再统一夹到 100–900 之间。转弯时还会走一个 `turn_mag(speed)`，若速度大于 150，再减 50，减少原地转向过猛的问题。

自动前进所用时长并不是固定 `forward_ms`，而是由 `step_length` 转成毫秒：

```
ms_calc = (steps << 7) - (steps << 1) - steps;
```

这一行等价于：

```
step_to_ms = step_length * 125
```

也就是说，系统把“一个逻辑步长”映射为 “125 ms × 步长值”的前进时间。于是答辩时你可以很自然地解释：

> “我们没有使用编码器闭环，而是采用步长到时间的标定映射。这样做的好处是实现简单、在 FPGA 上容易稳定复现；缺点是地面摩擦变化会带来累计误差，所以我们又保留了四轮校准参数和人工参数调节接口。”

手动模式还有一个很巧妙、也很“比赛友好”的设计：**只有手动前进会被避障阻断，手动后退和原地转向仍允许执行**。这是为了让车被障碍物顶住时，仍能靠“后退/转向”脱困，而不会整车完全锁死。

对应伪代码可写成：

```
if manual_mode:
    if obstacle_stop and manual_cmd == FORWARD:
        stop()
        motor_blocked = 1
    else:
        execute_manual_cmd()
elif obstacle_stop:
    stop()
    motor_blocked = 1
elif in_run and queue_not_empty:
    execute_queue_head_for_target_time()
```

C5 电机帧生成

`build/c5_motor_frame_driver.v` 是本项目最有“协议工程味道”的模块之一。它接收 4 路 11 bit 二补码有符号速度，转换成 C5 需要的 4 路脉宽值，再拼一个 ASCII 串口命令帧发出去。

左轮和右轮的脉宽映射分别是：

```
left_motor_pulse  = 1500 ± speed_abs
right_motor_pulse = 1500 ∓ speed_abs
```

换句话说：

```
左轮脉宽 = 1500 + signed_speed
右轮脉宽 = 1500 - signed_speed
```

这是因为左右轮机械定义方向相反，所以同样的“前进”需要相反的脉宽偏移方向。

更关键的是，这个模块不是“每拍都发”，而是有**发送调度与保护策略**：

- 上电先等待 500 ms
- 然后发送 3 帧 `#255PDST!` 停车命令
- 运动中每 100 ms 刷新一次运动帧
- 当命令变化、刷新到期、停止过渡或蓝牙强制停车发生时，才触发新的帧发送

这说明团队考虑到了**上电安全、串口带宽、运动指令稳定性和停车可靠性**，不是简单地把 UART 当“printf”在用。

你在答辩中可以主动突出这一点：

> “很多小车项目只关注速度值计算，我们额外做了发帧节流、上电停车序列和强制停机优先级，这能显著降低误动作风险。”

避障不是单点阈值，而是分层设计

超声波避障

`build/ultrasonic_obstacle_guard.v` 是一个标准分状态测距模块：

- `ST_TRIG`：输出 10 us 触发脉冲
- `ST_WAIT_ECHO`：等待回波上升沿，超时 30 ms
- `ST_MEASURE`：统计高电平持续时间，换算厘米
- `ST_GAP`：等待到总测量周期 60 ms，再开始下一次

距离换算近似为：

```
distance_cm = echo_cycles / 2900
```

其中 2900 来自 50 MHz 时钟条件下的声速经验换算。它的停车阈值并不是独立参数，而是直接拿 `step_length` 做比较长度；也就是**步长越大，超声波也越“保守”**。这是一种简洁但耦合较强的设计，答辩时可以把它说成是“统一了前进控制尺度与安全尺度”，同时也承认“若后续追求更精细控制，超声阈值应独立配置”。

RPLidar 避障

`build/rplidar_c1_uart.v` 是整个项目里算法感最强的模块。

它先做设备初始化：

1. 上电等待 0.5 s
2. 发送停止命令 `A5 25`
3. 再等 0.1 s
4. 发送普通扫描命令 `A5 20`
5. 等待 descriptor
6. 进入连续扫描解析

如果 2 s 没有收到有效字节，就认为离线，并进入重试流程。这个设计对评委很加分，因为它体现了**面向真实串口设备的容错思维**，而不只是“假设设备永远正常”。

收到扫描节点后，模块按 5 字节协议解析出：

- `quality`
- `angle_q6`，单位是 1/64 度
- `distance_q2`，单位是 1/4 mm

只接受：

```
quality != 0
50 mm <= distance <= 12000 mm
```

随后，它把 360° 划成 8 个 45° 扇区；每圈保留每个扇区的最小距离；再和前两圈历史值一起做**三样本中值滤波**。中值函数 `median3_q2(a,b,c)` 正是为了抑制偶发噪声、单圈误点和空扇区数据抖动。

伪代码如下：

```
for each valid lidar node:
    sector = angle_to_sector(angle)
    sector_min[sector] = min(sector_min[sector], distance)

at new_scan:
    filt_front = median3(front_min_cur, front_hist1, front_hist2)
    for s in 0..7:
        filt_sector[s] = median3(sector_min_cur[s], hist1[s], hist2[s])
```

真正体现“工程理解”的，是它后面的停车判定并不是只看正前方，而是按小车车体近似长方形来做：

- 正前方扇区：阈值 = `safe_distance_mm`
- 前左/前右扇区：阈值 = `safe_distance_mm + 150 mm`
- 左/右侧扇区：阈值 ≈ `220 mm`

代码中的常量正是：

- `CORNER_MARGIN_Q2 = 600` → 150 mm
- `SIDE_GUARD_Q2 = 880` → 220 mm

因此你在答辩中不要只说“我们用了雷达避障”，而应说：

> “我们把雷达避障做成了与车体形状相关的安全包络判断，不是单纯看前方一点距离。这样前角与侧向擦碰的风险也能提前拦住。”

这会显著提高你回答的技术含量。

所谓“传感器融合”到底是什么

当前仓库中的传感器融合并不是卡尔曼滤波，也不是贝叶斯融合。它实际上分两层：

1. **雷达内部融合**：当前圈最小值 + 前两圈历史值 → 三样本中值滤波
2. **雷达与超声波融合**：`obstacle_stop = lidar_stop OR ultrasonic_stop`

`build/obstacle_stop_merge.v` 甚至只有一句：

```
assign obstacle_stop = ultrasonic_stop | lidar_stop;
```

所以答辩时如果评委问“你们用了什么融合算法”，最稳妥的标准答案应是：

> “严格说不是概率融合，而是‘雷达时间域滤波 + 跨传感器安全逻辑合并’。这样做的目标是提高停车可靠性，而不是估计环境状态的最优值。”

蓝牙协议与状态回传

`build/bt_route_uart.v` 很值得讲，因为它不仅接收手机命令，还做了一个**统一的串口发送调度器**。其核心能力包括：

- 蓝牙心跳：收到 `P/p` 维持在线，3 s 无心跳判离线
- 手动模式切换：`H/h` 开手动，`O/o` 退回自动
- 手动动作：`W/A/S/D/C` 或同义键
- 参数调节：`G/N/B/+/-/V/T/1/2/3/4/5/Z`
- 雷达使能切换：`E/e` 开，`Q/q` 关
- 300 ms 手动动作看门狗：不续发则自动 STOP
- 短回复帧：`@Kx\n`、`@PONG\n`
- 长状态帧：`@S...` 路线/参数/C5 状态，`@L...` 传感器状态

最关键的工程点在于：**它没有为不同消息类型各开一路发送状态机，而是共用一个 TX 调度器**。当前实现会优先保证“短回复完整发完”，再发状态帧，避免回复帧插入大状态帧中间造成手机端解析错乱。这个设计非常像真正产品里的串口协议处理，而不只是比赛演示代码。

此外，`route_status_changed` 会检测路线、坐标、参数、手动状态、避障阻断等多种字段变化，一旦变化就立即置位 `route_status_pending`，不必死等下一个 200 ms 周期。因此它兼顾了**周期上报**与**事件触发上报**。

你在答辩中可以把这一段概括成：

> “我们把手机控制做成了一个闭环协议：手机不仅发命令，还能拿到路线、位置、参数、雷达、超声和电机帧状态；这让调试和比赛演示都更可控。”

HDMI 图形显示

`build/gui_route_renderer.v` 和 `build/hdmi_top.v` 组合起来，构成了项目的可视化前端。

这里没有帧缓存，也没有外部显存，而是**逐像素组合渲染**：

- `hdmi_timing_gen` 产生 1024×600 时序
- `gui_route_renderer` 根据 `pixel_x/pixel_y` 判断当前像素属于背景、状态栏、路线、当前位置、目标点、BT 状态还是文本
- `hdmi_tmds_encoder` 将 RGB888 编成 TMDS 10 bit
- `hdmi_serializer` 用 `ODDR + OBUFDS` 输出差分信号

`gui_route_renderer` 的一个非常有答辩价值的亮点是：它不是把路线当作一串图标画出来，而是会**根据历史命令和待执行命令重建线段列表**，然后再判断当前像素是否命中这些线段。这让“已走过路径”“剩余路径”“当前车头方向”和“目标点”都能在一张地图上同时显示。

但它也有一个非常明确的边界：模块内部 `ROUTE_WORDS = 8`，也就是**显示侧只展开 8 个命令槽**。因此当执行侧路线超过 8 个内部命令时，显示可能不完整。这是当前实现的一个真实限制，建议你在答辩时主动点出，而不是等评委问出来。

项目中没有的算法

为避免答辩时“被带节奏”，下面这张表建议你背熟：

常见高阶算法名词	仓库现状	应对说法
A* / Dijkstra 路径规划	没有	本项目路线由人通过按键/蓝牙编辑，不做全局自主规划
PID 速度闭环	没有	当前是定时开环控制 + 四轮静态校准
编码器闭环	有旧模块，但当前顶层未使用	旧链路保留，当前版本未接入主链
Kalman / EKF 融合	没有	采用中值滤波 + 逻辑合并，更适合当前赛题与实现周期
视觉/图像处理	没有摄像头输入	HDMI 仅做状态显示，不做视觉感知
机器学习模型	没有	当前重点是 FPGA 外设并行控制与可靠闭环

如果评委问到了这些，而你又没有做，一句真诚且专业的话就足够：

> “这版项目没有强行上高阶算法，我们优先保证了系统闭环、外设稳定和比赛可演示性。若后续迭代，再考虑引入视觉或闭环控制。”

开发时间线、难点与问题复盘

先说结论

由于 GitHub commit history 无法直接获取，这一节不是“真正的 commit 列表解析”，而是依据工程内 `backup/*.zip`、`hdmi_rgb_input/`、`model/`、`build/` 文件时间戳重建出来的**开发时间线**。它的优点是相当接近真实演化；缺点是不能看到每次 commit message 的原始文字。这个限制建议你在向老师/队友汇报时也说明清楚。 [3]

重建出的开发阶段

时间	证据	推断阶段	可能的开发重点
2026-04-18	`backup/416_hdmi/`、`419with sys.zip`	HDMI + GUI + 早期路线状态机原型	先把显示、时钟和基础路线编辑跑通
2026-04-19	`420 all.zip`	引入 GPIO、旧电机链、`route_contol_core`	从“会显示”走向“会控制车”
2026-04-20	`420allless.zip`、模型时间戳	补 buzzer、GUI、路线核心细化	提升交互反馈与模块完整性
2026-04-21 到 04-24	`424full.zip`、`424lastmaybe.zip`	加强约束、顶层集成、旧底盘方案成熟	首次形成完整小车工程形态
2026-07-07	`hdmi_rgb_input/key_low_merge.v`、`route_gui_fsm_top.v`、`c5_motor_test_pattern.v`	重新整理主链，准备切向 C5 小车执行链	输入整合、顶层重构
2026-07-09	`change/top.xdc`、`rplidar_c1_uart.model`、`car_param_controller.model`	接入雷达、参数控制、更新引脚和模型备份	感知链与参数配置上线
2026-07-11	`bt_route_uart.model`、`c5_route_controller.model`	蓝牙协议与新运动控制器基本定型	手机控制、自动/手动统一
2026-07-12	`hdmi_rgb_input/bt_route_uart.v`、`c5_route_controller.v`、`c5_motor_frame_driver.v`	导出关键主模块 Verilog	准备综合实现
2026-07-15	`build/*.v`、`top.json/fasm/frame/bit`	当前最终导出与 bitstream 生成	项目收束、可下载版本形成

这条时间线最有价值的地方，在于它非常清楚地显示了团队的开发顺序不是“一口气全做”，而是典型的工程节奏：**先显示和状态，再控制，再硬件接口，再手机协议，再传感器，再整体打包。** 这正是你答辩时可以强调的工程方法论。

代码中能看出的主要难点与对应解决思路

路线核心从旧版到新版的迁移

工程里同时保留了旧的 `sys_route_control_core` 和当前实际使用的 `route_contol_core`。旧版核心直接在 64 bit `route_info` 上移位，并用 `step_start/step_words` 记录撤销边界；新版改成了 `route_cmd_mem[]` 与 `old_route_cmd_mem[]` 两组命令内存，更容易做历史显示和队首出队。

这说明一个明显的开发难点是：**如何让路线既能编辑/撤销，又能在运行时逐条出队，还要给 GUI 保留历史路径。** 当前版本通过“活动队列 + 历史队列”把这件事拆开了，这是合理的。

从 TB6612 旧链路切到 C5 小车链路

仓库里存在一整套旧模块：`motor_encoder_interface`、`motor_route_controller`、`motor_tb6612_driver`。而当前顶层真正使用的是：`c5_route_controller` + `c5_motor_frame_driver`。这通常意味着团队在开发过程中经历过**底盘硬件方案切换**，从本地 PWM/TB6612 风格的驱动，切换到通过串口协议控制 C5 小车。

这种切换会带来两类典型 bug：

- 原来的“速度 = PWM 占空比”思维不再成立，必须改成“速度 = 脉宽/串口帧”
- 原来电机控制是低层实时输出，现在变成“指令帧调度”，必须重新考虑停车、上电和刷新周期

当前 `c5_motor_frame_driver` 中完整的“500 ms 等待 + 3 次停车帧 + 100 ms 刷新”明显就是解决这类问题后的产物。

蓝牙发包互相打断的问题

从 `bt_route_uart.v` 的结构看，当前实现采用了唯一 TX 调度器，并严格区分 `pending_reply`、`sending_reply`、`sending_status`。这几乎可以确定团队在开发中踩过一个坑：**短回复、心跳回包和长状态帧可能相互穿插，导致手机解析端收脏数据。**

当前版本的解决思路是非常标准的工程做法：把“发什么”与“怎么串口发送”解耦，只保留一个物理 TX 状态机，让消息在高层排队。

如果评委问“你们调试过程中遇到的协议问题是什么”，这是一个非常好的真实案例。

设备启动时序问题

雷达与 C5 驱动都包含明显的启动保护逻辑：

- 雷达：Boot delay → Stop → Wait → Scan → Descriptor → Continuous data
- C5：上电等待 → 停车帧 → 正常运动

这通常意味着真实硬件不是“上电即用”，而是存在**上电扰动、旧状态残留、串口同步丢失**之类问题。当前版本用显式状态机把时序写死，说明这是一个被解决过的关键点。

当前快照中的真实缺陷或技术债

这一节建议你直接加入自己的技术备忘，因为它们很可能在答辩问答里出现。

问题	位置	影响	建议说法
路线存储边界不一致	`route_contol_core.v`	16 项数组 vs 部分 32 上限判断，潜在越界	“已识别，当前版本以 16 项真实边界为准，后续统一修正判断条件”

问题	位置	影响	建议说法
GUI 仅显示 8 个命令槽	`gui_route_renderer.v`	长路径显示不完整	“执行链支持更长命令，显示链当前只渲染前 8 槽，是界面限制不是控制限制”
转向也计入路线长度条	`route_contol_core.v`	长度显示偏控制量，不是纯几何长度	“当前长度指标用于控制进度展示，若追求物理里程会拆分前进/转向统计”
急停后清空活动路线	`route_contol_core.v`	不能直接 resume	“当前优先保证安全，障碍停后要求重新确认路线，属于保守设计”
超声阈值与步长耦合	`ultrasonic_obstacle_guard.v`	参数耦合较大	“当时为了简化调参统一了尺度，后续可独立化”
C5 反馈未闭环接入	顶层 `c5_uart_rx` 未使用	无法读取执行器反馈	“当前是发送型控制链，保留了后续反馈扩展接口”

你要有意识地把这些“潜在弱点”转化为“我已经识别并理解的设计边界”。评委最怕的不是项目有边界，而是作者自己不知道边界在哪里。

往年集创赛与 Robei 赛道答辩规律

赛事形态与答辩关注点

公开可检索资料表明，集创赛长期采用**初赛—分赛区决赛—全国总决赛**的三级赛制；总决赛常见的评分构成包括**演讲汇报、作品展示、问答**，而分赛区/总决赛都高度重视事先提交的**技术文档与答辩 PPT**。官方通知还明确说明：技术文档和答辩 PPT 是重要评分依据，内容可包括作品技术方案、实现逻辑、重要代码与仿真截图等。[7]

多所高校的校内赛/选拔赛通知给出的典型时间分配是：**8 分钟以内答辩 + 5 分钟以内提问**，或 **10 分钟总时长，其中约 6 分钟 PPT 展示 + 4 分钟评委提问**。虽然这不等于每个分赛区/杯赛都完全相同，但它足够说明一个事实：**你的表达必须高度压缩，且演示链路必须一镜到底。** [8]

Robei 赛道公开线索

公开资料显示，Robei 官网曾明确提到 Robei 杯/赛道的总决赛带有**实操比赛**属性，且与**可重构机器人平台、蓝牙控制、创意展示**密切相关；另一些高校或培训通知则显示 Robei 相关赛题还可能围绕 Robei EDA 工具、数字 IP 或 FPGA 应用设计展开。这说明 Robei 方向的赛题具有明显的**企业命题迭代性**，但共同点始终是：**强调 Robei 平台上的数字系统设计、工程实现与作品可展示性**。 [9]

公开视频检索中还能看到类似“第五届集创赛 Robei 杯国赛三等奖演示视频”“华中赛区集创赛答辩预演记录版”这样的材料。这类题名本身就说明，往届队伍通常把重点放在**功能演示闭环、作品动作展示、答辩预演打磨**上，而不仅是展示原理图。 [10]

从官方规则反推的高频评委问题

结合官方评分要素、高校赛前经验分享和公开演示视频标题，评委高频关注点大概率集中在以下七类。这一部分虽然属于归纳推断，但推断基础是可靠的：赛事评分强调文档/PPT/演示/问答，高校经验分享强调“关键环节和常见问题”，而 Robei 赛道又天然偏工程实现。 [11]

评委关注点	他们真正想听什么	你这个项目的最佳切入点
赛题理解	你到底解决了什么问题	“基于 FPGA 的路线编辑、执行、避障和可视化闭环小车系统”
创新点	不是拼功能，而是拼“为什么这样设计”	“多外设并行、协议统一、HDMI 可视化、参数可在线调、双传感器安全停车”
关键技术	是否真正做了核心模块	讲 `route_contol_core`、`c5_route_controller`、`rplidar_c1_uart`
验证方法	你怎么证明它能工作	讲 testbench、波形、HDMI 界面、蓝牙状态帧、现场演示
指标与取舍	为什么不用更复杂算法	讲赛题目标、开发周期、FPGA 实现复杂度、稳定性优先
工程边界	你知道自己没做什么吗	主动承认无 PID/SLAM/视觉/ML，当前为开环 + 安全停车
后续迭代	你有没有继续优化路线	讲边界统一、编码器闭环、C5 反馈接入、路径恢复、独立安全阈值

对你最有用的答辩策略判断

就你的项目而言，最容易拿分的不是“算法先进”，而是下面这三点：

第一，把**系统闭环**讲完整：
输入怎么进来，路线怎么存，动作怎么执行，停车怎么触发，状态怎么回显。

第二，把**工程约束与取舍**讲清楚：
为什么现在是开环定时控制而不是 PID；为什么超声和雷达是这种融合方式；为什么要做唯一 TX 调度器。

第三，把**你知道的问题**提前说出来：
容量边界、GUI 显示槽数、急停后清空路线、长度条是控制量等。
评委往往不会因为项目有边界扣很多分，但会因为作者回避细节而显著减分。

面向答辩的讲解稿与问答题库

面向答辩的项目讲解稿

下面这份讲解稿按 **约 6 分钟主讲 + 4 分钟问答** 的节奏写，适合你直接改成 PPT 备注稿。

开场

各位老师好，我们的项目是一个基于 Robei EDA 与 FPGA 的小车控制系统。它实现了**路线编辑、自动执行、蓝牙手动控制、双传感器避障、HDMI 实时显示和状态回传**。我们想解决的核心问题不是“做一个会动的小车”，而是“在 FPGA 上把多外设、多协议和多状态机可靠地集成成一个完整闭环系统”。

任务和整体思路

项目整体分成五层。第一层是输入层，包括板载五向按键和手机蓝牙命令；第二层是路线与参数层，负责保存路线队列、当前状态和可调参数；第三层是运动控制层，把离散命令转换成四轮速度；第四层是感知层，用超声波和 RPLidar 做安全停车；第五层是可视化层，通过 HDMI 和蓝牙把系统状态实时展示出来。

关键模块

路线核心在 `route_contol_core`。我们把路线保存成 2 bit 的离散命令序列，`00` 停止、`01` 右转、`10` 左转、`11` 前进。这样做的好处是执行逻辑非常简单，而且很适合和 FPGA 状态机配合。用户按左或右时，系统内部会自动展开成“转向 + 前进一步”，因此路线既能撤销，也能逐条出队执行。

运动控制在 `c5_route_controller`。这个模块不是 PID，而是时间驱动的开环控制器。前进时间由 `step_length` 转换而来，转向时间使用固定 `turn_ms`。同时我们给四个轮子都留了默认速度修正量，用来抵消机械一致性误差。这样在赛题约束下更容易调试，也更稳定。

感知层包括两个模块。超声波模块做前向近距离测距；雷达模块会把 360 度扫描划分为 8 个扇区，每个扇区取最小值，并对连续三圈做中值滤波。我们不是只看正前方一个点，而是按小车近似长方形轮廓设置前方、前角和侧面阈值，因此停车判断更稳健。

我们还做了一个比较完整的手机协议模块 `bt_route_uart`。手机不仅能发控制命令，还能收到路线状态、位置、参数、雷达和电机状态。我们专门做了唯一的串口发送调度器，避免心跳回复和大状态帧相互打断，保证 APP 解析稳定。

最后，`gui_route_renderer + hdmi_top` 会把状态栏、路线网格、当前位置和目标点实时显示到 HDMI 屏幕上。这样现场演示时，评委能同时看到小车动作和内部状态变化，不需要猜系统当前在做什么。

项目亮点

我们的亮点主要有三点。
第一，系统闭环完整，不只是控制，还包括显示、回传和安全停车。
第二，模块边界清晰，主链路从输入、状态机、控制器到执行器是层层解耦的。
第三，工程可调试性强，蓝牙状态帧、HDMI 画面、testbench 和 bitstream 产物都能作为验证证据。

真实边界与后续优化

当前版本也有边界。我们没有做自动路径规划、PID 或视觉识别；路线显示模块当前只渲染前 8 个命令槽；障碍停车后采取的是保守策略，会清空当前活动路线；超声波阈值和步长参数目前耦合。下一步我们计划优先完善边界判断统一、加入编码器或反馈闭环、扩展 GUI 路线显示槽数，并考虑支持急停后的路线恢复。

收尾

总体来说，我们希望展示的是：在 Robei/FPGA 环境下，如何把“路线编辑—运动执行—避障停车—状态可视化”做成一个真正可演示、可调试、可迭代的系统，而不只是零散功能拼接。谢谢各位老师。

建议的演示流程

演示顺序	你做什么	评委看到什么	你要说什么
开机	给板上电	HDMI 亮起，蜂鸣提示	“系统上电后先做内部复位和停车保护”
进入编辑态	用按键或手机加路线	HDMI 路线变化	“路线按离散命令保存，左/右会自动展开为转向加前进”
调参数	用手机调步长/速度/激光阈值	HDMI/蓝牙状态变化	“参数在线可调，不需要重新综合”
自动运行	启动路线	小车动作 + HDMI 当前/目标位置变化	“每次动作完成都有 `run_step_pulse` 推动队列出队”
障碍演示	在前方放障碍	小车停车、状态帧显示阻断	“避障由超声和雷达共同约束，控制器进入停车逻辑”
手动脱困	手机后退/转向	前进被阻断，但后退/转向可执行	“这是为了被障碍顶住后还能人工脱困”
状态回传	展示 APP 或串口日志	`@S/@L` 数据变化	“手机端可以看到路线、参数、传感器和电机帧状态”

答辩问答题库

基础问题

难度	问题	示范回答
基础	这个项目最核心的功能是什么	核心是把路线编辑、自动执行、蓝牙控制、双传感器避障和 HDMI 可视化整合成一个 FPGA 小车闭环系统。
基础	路线是怎么保存的	路线按 2 bit 离散命令保存，`00` 停止、`01` 右转、`10` 左转、`11` 前进，执行时从低位命令开始出队。
基础	左转为什么要展开成两条命令	因为执行层更容易处理“先改朝向，再前进一步”，这样撤销、显示和执行都统一。
基础	为什么要做 HDMI 显示	现场调试时只看小车动作不够，HDMI 能把路线、当前位置、目标点和连接状态直观显示出来，评委也更容易理解。
基础	蓝牙能做哪些事情	能切自动/手动模式、发运动命令、调参数、开关雷达，并接收路线状态和传感器状态。
基础	你们用了哪些传感器	当前是超声波和 RPLidar C1，两者共同参与停车约束。

进阶问题

难度	问题	示范回答
进阶	你们的路径规划算法是什么	严格说没有自动路径规划。当前项目采用人工编辑的离散路线队列，优势是实现简单、行为可解释、适合 FPGA 状态机实现。
进阶	你们的控制器是不是 PID	不是。当前是开环定时控制加四轮静态速度校准。前进时间由步长映射，转向时间固定，重点是系统稳定和可调试。
进阶	四个轮子的速度为什么不一样	因为机械装配和摩擦会导致一致性误差，所以我们给每个轮子单独保留默认速度修正量，再和整体速度组合。
进阶	避障为什么既要超声又要雷达	超声波适合近距离前向停车，雷达适合更大范围感知和方向性判断。两者结合能同时兼顾近距可靠性和空间信息。
进阶	雷达数据怎么滤波	先按 8 个扇区取当前圈最小值，再和前两圈历史值做三样本中值滤波，减少单圈噪声和偶发异常点影响。
进阶	为什么前进被阻断时还能后退	这是刻意设计的脱困策略。手动前进遇障碍会停，但后退和原地转向仍允许，避免整车完全锁死。

高阶问题

难度	问题	示范回答
高阶	这套系统最大的技术风险是什么	当前最大的技术债有三个：路线核心边界判断尚未完全统一、GUI 只显示前 8 个命令槽、急停后不支持直接恢复。这些都已识别，优先级也明确。
高阶	路线长度统计是否准确	当前长度条更偏向控制步数指标，因为转向命令也会计入 `steps * step_length`。如果要表示准确物理路径，应把前进和转向分开统计。
高阶	你们为什么不用编码器闭环	旧链路里保留了相关模块，但当前主版本为了确保赛期内稳定落地，优先采用开环定时 + 参数标定方案。后续迭代可以把反馈闭环接进来。
高阶	蓝牙长状态帧会不会和短回复打架	我们专门做了唯一 TX 调度器，回复帧和状态帧不会并行发送，避免 `@PONG` 之类插到 `@S/@L` 中间。
高阶	障碍停车后为什么直接清空路线	当前采用保守安全策略，避免障碍消失后无条件继续执行造成风险。后续可以扩展为“暂停/恢复”双模式。
高阶	你们的创新点到底是什么	不是单一算法创新，而是把 Robei/FPGA 下的路线状态机、参数在线调节、双传感器安全约束、手机协议和 HDMI 可视化整合成完整工程闭环。

刁钻问题

难度	问题	示范回答
刁钻	如果评委认为这不够“智能”怎么办	我会先承认当前版本不是以智能规划为目标，而是以可靠控制闭环为目标；然后强调 FPGA 系统集成、协议调度和可视化验证本身就是本项目的技术重点。

难度	问题	示范回答
刁钻	如果实际地面摩擦变化很大，开环方案怎么办	这是开环方案的已知边界。我们当前通过步长标定和四轮修正缓解误差，下一阶段会优先引入反馈闭环或编码器修正。
刁钻	GUI 为什么只显示 8 段，你们没发现吗	发现了。执行核心和显示核心的槽数目前确实不一致，执行能覆盖更长命令，显示是为当前屏幕布局做的折中版本，这是后续优先修复项。
刁钻	你们说有 commit history 分析，真实 commit 在哪	本次能直接核验的是代码快照。由于在线仓库页面不可访问，没法直接给出 GitHub commit DAG，所以我在报告里把时间线明确标注为“依据工程快照重建”。

技术备忘与临场建议

详细技术备忘

文件/函数索引表

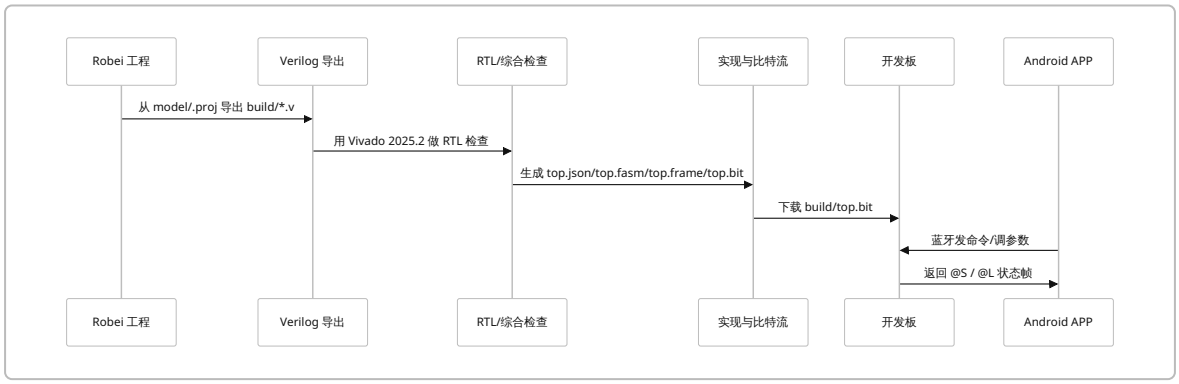
下表是建议你打印或存在平板里的“速查版”。

文件	你该记住的作用	关键端口/变量/函数	速查用途
`build/sys_route_gui_fsm_top.v`	当前总顶层	`obstacle_stop_merge20_obstacle_stop`、`bt_route_uart16_*`	讲全系统连接时先看它
`build/sys_route_control_fsm.v`	路线状态机包装器	5 个 `key_pulse_filter` + `route_contol_core`	解释按键消抖与单脉冲
`build/route_contol_core.v`	路线编辑/运行核心	`route_cmd_mem`、`old_route_cmd_mem`、`move_*` 任务	讲路线机制必看
`build/car_param_controller.v`	参数管理	`overall_speed`、`step_length_set`、`lidar_safe_mm`	讲调参数必看
`build/c5_route_controller.v`	运动控制	`adjusted_speed()`、`turn_mag()`、`step_to_ms()`	讲“控制器”必看
`build/c5_motor_frame_driver.v`	C5 串口驱动	`left_motor_pulse()`、`right_motor_pulse()`、`frame_char()`	讲执行器协议必看
`build/ultrasonic_obstacle_guard.v`	超声波避障	`ST_TRIG/WAIT_ECHO/MEASURE/GAP`	讲超声时序必看
`build/rplidar_c1_uart.v`	雷达解析与避障	`median3_q2()`、`node_sector`、`obstacle_stop`	讲雷达必看

文件	你该记住的作用	关键端口/变量/函数	速查用途
`build/obstacle_stop_merge.v`	停车合并	`ultrasonic_stop | lidar_stop`	非常简单但经常会被问到
`build/bt_route_uart.v`	手机协议与回传	`cfg_command()`、`reply_char()`、`status_char()`	演示 APP 协议时必看
`build/gui_route_renderer.v`	HDMI 画面生成	`calc_step_pix()`、路线线段数组	讲可视化与资源占用必看
`build/hdmi_top.v`	HDMI 管线	`clk_wiz_0`、`hdmi_tmds_encoder`、`hdmi_serializer`	讲显示输出时必看
`build/clk_wiz_0.v`	时钟产生	`MMCME2_ADV`、`BUFG`、`stable_locked`	讲时钟与复位时必看
`build/power_on_reset.v`	上电复位	`POR_CYCLES = 1000000`	解释上电稳定性
`source_review/KEY_VARIABLES.md`	编码与参数速查	状态编码、参数范围、协议字段	答辩前快速复习
`test/README_experiments.md`	验证入口说明	test 名单与关注点	评委问“怎么验证”时可引用

构建与部署步骤

基于当前工程快照，建议把部署流程讲成下面这样：



如果评委问“你们怎么构建”，你可以回答：

1. 在 Robei 中维护 `model/` 和 `little_car.proj` 工程

2. 导出当前顶层对应的 `build/*.v`
3. 通过附带的脚本用 Vivado 2025.2 做 RTL 级检查
4. 生成 `top.bit` 并下载到 Artix-7 目标板
5. 接上蓝牙、雷达、超声和 HDMI 外设进行联调

其中，Vivado 2025.2 是可以直接从 `source_review/verify_sources.bat` 看到的；而实现具体脚本未随快照附带，所以你不要把“实现工具详情”说得过死，最好表述成“当前快照已含 bitstream 产物，具体实现脚本在工程外部环境中完成”。这样更稳。

测试覆盖面怎么答

仓库里已经有一批 testbench 是可以直接拿来当答辩证据的：

测试文件	覆盖内容	你可以怎么说
`test/c5_motor_frame_driver_robei_test.test`	上电保护、停车序列、UART 字节、刷新周期	“执行层不是盲发，时序做过验证”
`test/c5_motor_mapping_robei_test.test`	前后左右与四轮脉宽映射	“四轮映射关系做过单独检查”
`test/car_param_controller_robei_test.test`	默认值、增减与上下限	“参数面板不是拍脑袋，是有边界的”
`test/c5_route_controller_robei_test.test`	自动/手动/避障阻断	“控制器的优先级和阻断逻辑做过验证”
`tb/tb_route_contol_core_fast.v`	路线核心快速行为	“路线编辑规则可仿真验证”
`tb/tb_ultrasonic_obstacle_guard_robei.v`	超声时序/超时	“传感器时序不是只在板上试”

如果你没有现成波形图，至少把 testbench 名字和覆盖范围写进备份 PPT。这样评委一问“如何验证”，你不会只剩一句“我们在板子上试过”。

答辩注意事项与临场应对建议

你一定要主动说的四句话

第一句：
“这个项目的重点是 FPGA 多模块系统闭环，不是单点算法炫技。”

第二句：
“我们当前采用的是开环定时控制 + 四轮参数校准，而不是 PID 闭环。”

第三句：
“雷达避障是扇区最小值加历史中值滤波，再结合车体安全包络判断。”

第四句：
“当前版本的边界我们是清楚的，例如路线显示槽数、急停后恢复、容量判断统一性等。”

只要你提前把这四句话说出来，评委再追问时，你会始终处于主动位置。

遇到不会的问题怎么“机智应对”

最忌讳的方式是硬编。最推荐的是三段式回答法：

先承认边界
“这个点我们当前版本没有做到完全闭环……”

再说明现状
“目前项目里采用的是开环定时加参数标定，原因是赛期内要优先保证整体系统稳定。”

最后给后续方案
“如果继续迭代，我们会优先把编码器反馈接入 `c5_route_controller`，把它从时间驱动升级成带反馈修正的控制器。”

这套说法几乎适用于所有“你们为什么没做 XXX”的问题。

如果评委追问“创新点不够强”怎么办

你不要和评委争“我们已经很创新了”，而要把创新点重新定义到工程层：

> “我们的创新点不在单个算法，而在于把路线编辑、双传感器安全约束、手机协议、HDMI 可视化和 C5 串口驱动做成可运行、可调、可展示的一体化 FPGA 系统。对于 Robei/FPGA 类赛题，这种集成能力本身就是很关键的能力体现。”

最后的临场小抄

场景	你该怎么做
讲超声波时	一定说清 10us 触发、60ms 周期、30ms 超时
讲雷达时	一定说清 8 扇区、三样本中值滤波、不是单点阈值
讲控制器时	一定说清“不是 PID，是开环定时控制 + 标定”
讲路线时	一定说清“左/右会展开成转向 + 前进”
讲 APP 时	一定提 `@S/@L` 状态帧和唯一 TX 调度器
被问 GitHub 历史时	直接说明在线仓库页面不可访问，本次报告的时间线是依据工程快照重建
被问未来工作时	优先答：边界统一、GUI 扩展、反馈闭环、急停恢复、独立安全阈值

如果你只能在答辩前最后复习一页，请复习这一页。它决定的是你答辩时的“稳定度”，而不是知识点数量。

最适合你项目的收尾话术

答辩最后 20 秒，建议你这样总结：

> “这版小车项目最重要的不是某一个炫目的算法，而是把 FPGA 上的状态机、协议、传感器、执行器和可视化真正闭合起来。我们对当前版本的优势、边界和后续优化方向都比较清楚，因此它既是一个可展示的比赛作品，也是一个可持续迭代的工程底座。”

---

1 3 Internal Error

2 11 重要通知- 集创赛
https://univ.ciciec.com/nr.jsp?groupId=2&jpt=4&utm_source=chatgpt.com

4 Robei 3.5.1 X64最新版下载
https://www.robei.com/download.php?utm_source=chatgpt.com

5 全国大学生集成电路创新创业大赛-Robei杯...
https://robei.com/read.php?id=150&utm_source=chatgpt.com

6 Robei EDA
https://robei.com/?utm_source=chatgpt.com

7 关于组织学生参加2022年第六届全国大学生集成电路创新创业大赛 ...
https://mece.bjtu.edu.cn/cms/item/3393.html?utm_source=chatgpt.com

8 关于第七届全国大学生集成电路创新创业大赛“希荻 ...
https://gr.uestc.edu.cn/tongzhi/118/11524?utm_source=chatgpt.com

9 概要
https://robei.com/listarticle.php?cat=news&utm_source=chatgpt.com

10 集创赛-哔哩哔哩
https://search.bilibili.com/all?keyword=%E9%9B%86%E5%88%9B%E8%B5%9B&utm_source=chatgpt.com